

FEDERAL UNIVERSITY
OF SANTA CATARINA

EEL5105 – Circuitos e Técnicas Digitais
Aula 8

Prof. Héctor Pettenghi

hector@eel.ufsc.br
http://hectorpettenghi.paginas.ufsc.br

Material desenvolvido com apoio de arquivos de apresentação do livro de Frank Vahid

8. Projeto de Circuitos Sequenciais

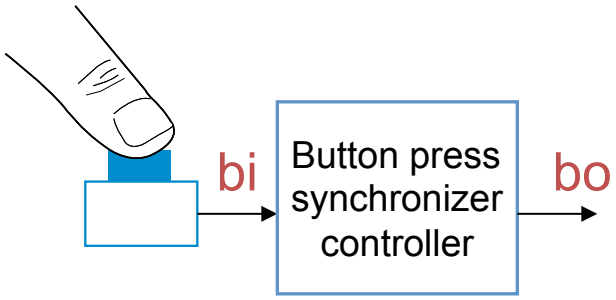
8.1. Projeto de FSM (continuação)

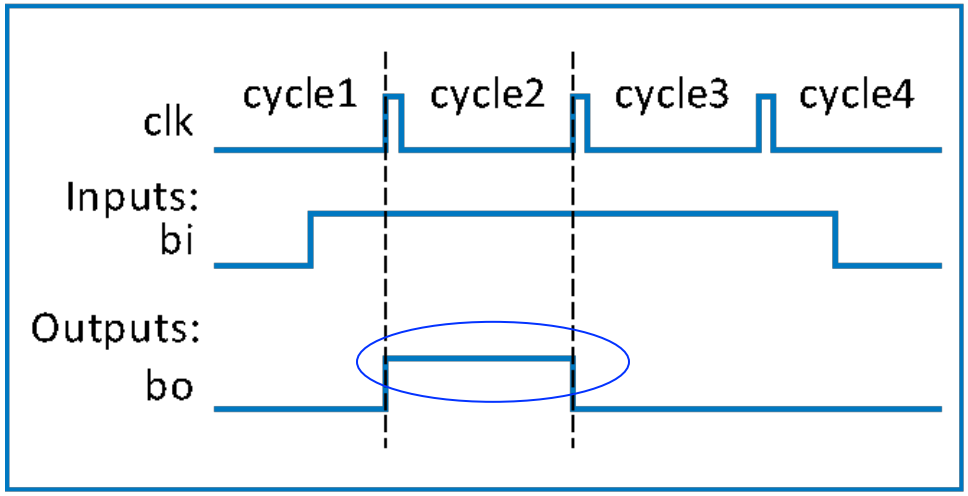
8.2. FSM com estados de bloqueio (Lock-out)

8.3. Engenharia reversa de projeto FSM

8.1. Projeto de FSM (continuação)

- **Exemplo 1:** Projetar um sincronizador de aperto de botão (circuito que converte um toque no botão em um pulso com duração de um ciclo de *clock*, independentemente do tempo que o botão fica apertado).



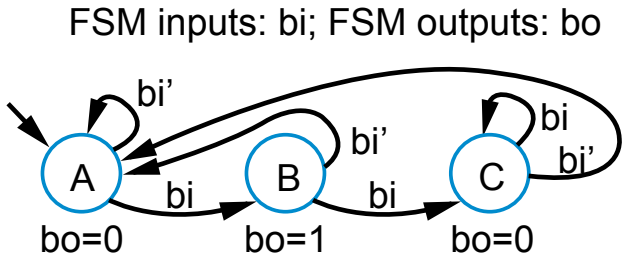


8.1. Projeto de FSM (continuação)

- **Exemplo 1:** Projetar um sincronizador de aperto de botão (circuito que converte um toque no botão em um pulso com duração de um ciclo de *clock*, independentemente do tempo que o botão fica apertado).
  - **Especificação formal:** ***Capturar a FSM***

8.1. Projeto de FSM (continuação)

- **Exemplo 1:** Projetar um sincronizador de aperto de botão (circuito que converte um toque no botão em um pulso com duração de um ciclo de *clock*, independentemente do tempo que o botão fica apertado).
  - **Especificação formal:** ***Capturar a FSM***

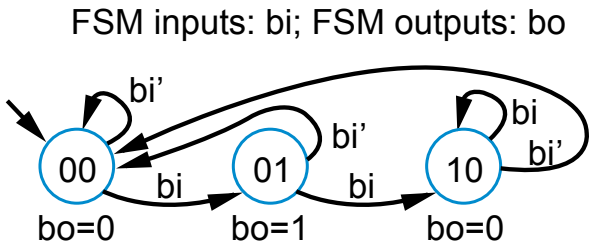


8.1. Projeto de FSM (continuação)

- **Exemplo 1:** Projetar um sincronizador de aperto de botão (circuito que converte um toque no botão em um pulso com duração de um ciclo de *clock*, independentemente do tempo que o botão fica apertado).
  - **Passo 1:** ***Codificar os estados***

8.1. Projeto de FSM (continuação)

- **Exemplo 1:** Projetar um sincronizador de aperto de botão (circuito que converte um toque no botão em um pulso com duração de um ciclo de *clock*, independentemente do tempo que o botão fica apertado).
  - **Passo 1:** ***Codificar os estados***



Codificação binaria

	s1	s0
A	0	0
B	0	1
C	1	0

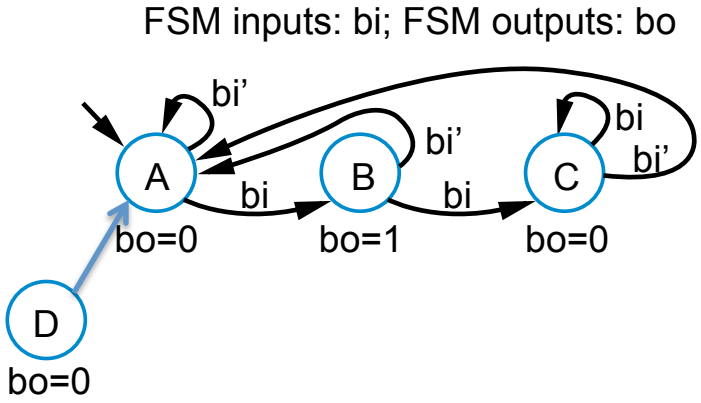
  - **Passo 2:** ***Criar uma tabela de estados***

8.1. Projeto de FSM (continuação)

- **Exemplo 1:** Projetar um sincronizador de aperto de botão (circuito que converte um toque no botão em um pulso com duração de um ciclo de *clock*, independentemente do tempo que o botão fica apertado).
  - **Passo 2:** ***Criar uma tabela de estados***

	s1	s0	bi	n1	n0	bo
A	0	0	0	0	0	0
	0	0	1	0	1	0
B	0	1	0	0	0	1
	0	1	1	1	0	1
C	1	0	0	0	0	0
	1	0	1	1	0	0
	1	1	0	—	—	—
	1	1	1	—	—	—

Não usado (estado D)



Don't cares podem ser preenchidos com "0"s

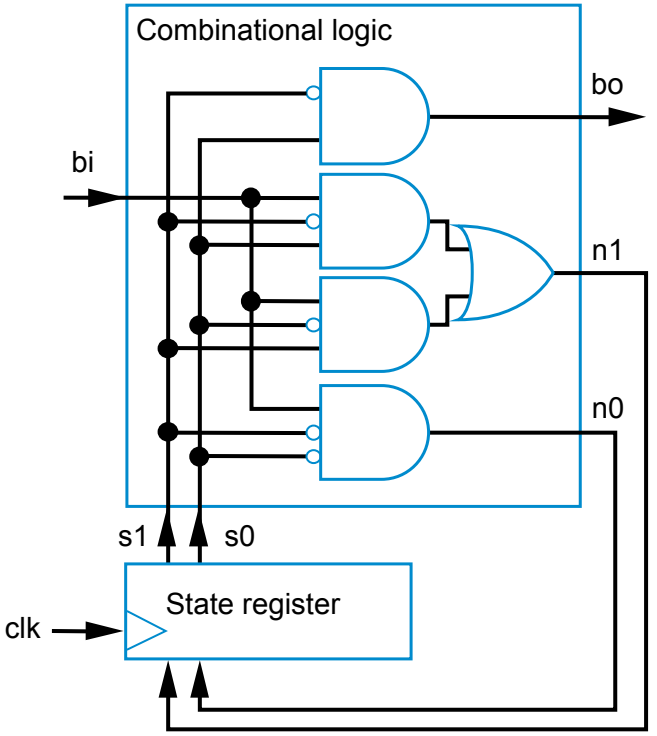
8.1. Projeto de FSM (continuação)

- **Exemplo 1:** Projetar um sincronizador de aperto de botão (circuito que converte um toque no botão em um pulso com duração de um ciclo de *clock*, independentemente do tempo que o botão fica apertado).
  - **Passo 3:** ***Implementar...***

8.1. Projeto de FSM (continuação)

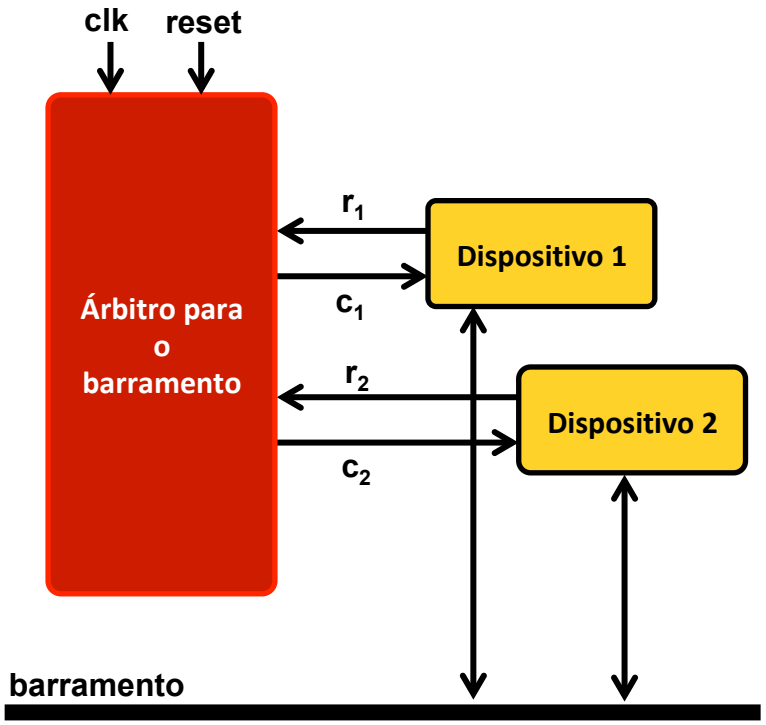
- **Exemplo 1:** Projetar um sincronizador de aperto de botão (circuito que converte um toque no botão em um pulso com duração de um ciclo de *clock*, independentemente do tempo que o botão fica apertado).
  - **Passo 3:** ***Implementar...***

n1 = s1's0bi + s1s0'bi
n0 = s1's0'bi
bo = s1's0bi' + s1's0bi = s1's0



8.1. Projeto de FSM (continuação)

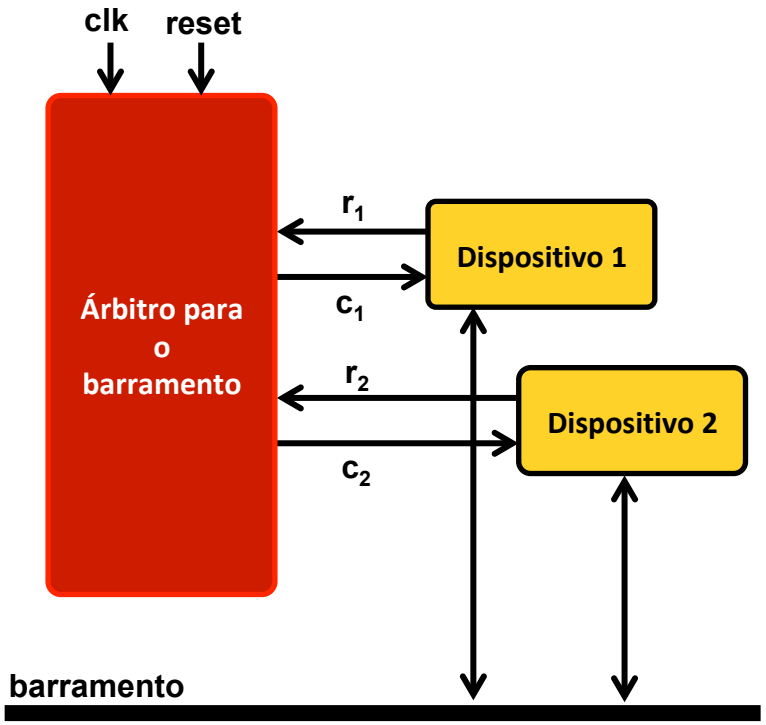
- **Exemplo 2**: Faça o projeto de um circuito árbitro para o acesso a um barramento que funcione conforme descrito a seguir.



- O árbitro recebe requisições para o uso do barramento de 2 dispositivos diferentes
- O "Dispositivo 1" tem prioridade sobre o "Dispositivo 2" para o uso do barramento
- Somente um dispositivo por vez pode ser autorizado a usar o barramento
- Para sinalizar que o dispositivo pode usar o barramento, o árbitro faz o sinal de concessão igual a 1 (c1 = 1 ou c2 = 1, dependendo do dispositivo)

8.1. Projeto de FSM (continuação)

- **Exemplo 2**: Faça o projeto de um circuito árbitro para o acesso a um barramento que funcione conforme descrito a seguir.



• Uma vez que o dispositivo recebe a concessão para usar o barramento, ele permanece com esta concessão durante todo o tempo que ele necessitar usar o barramento (por isso, o dispositivo mantém seu sinal de requisição em nível alto durante o período de uso do barramento)

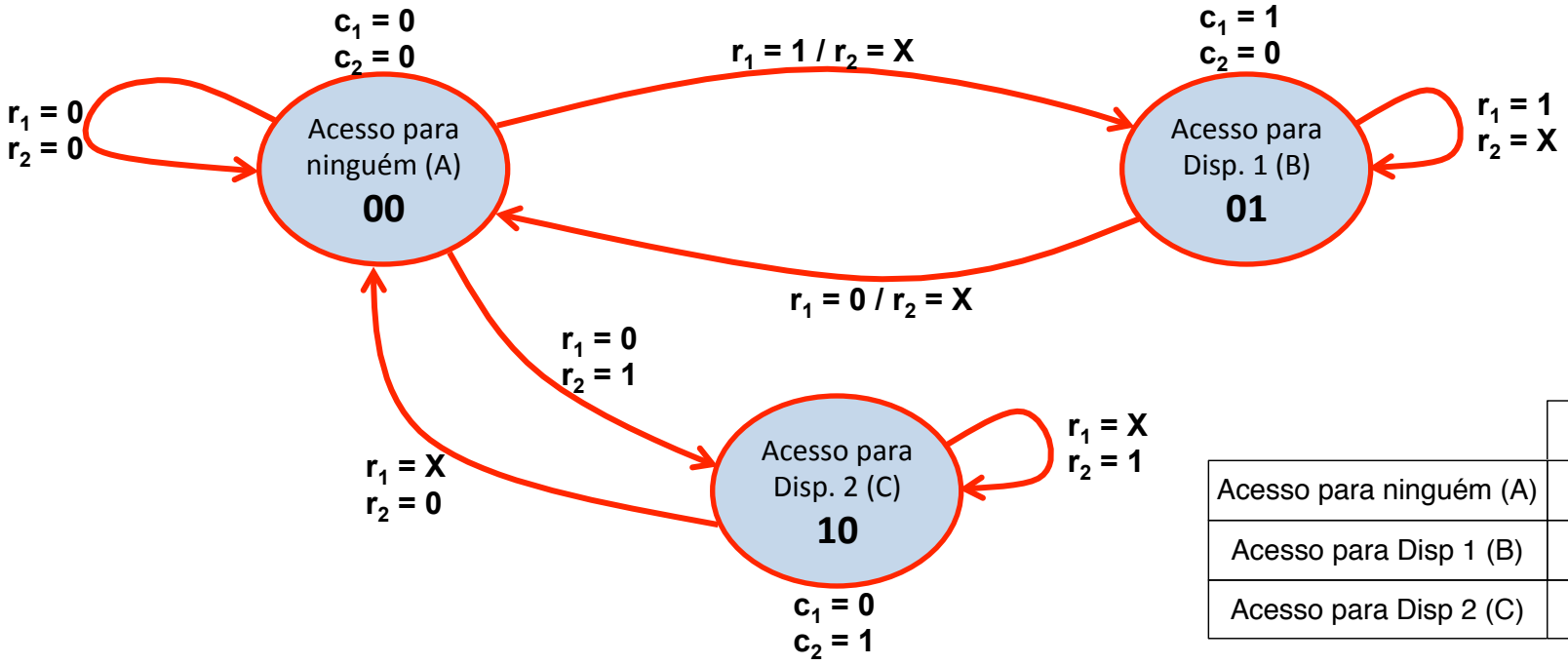
• Faça o projeto do árbitro...

8.1. Projeto de FSM (continuação)

- **Exemplo 2**: Faça o projeto de um circuito árbitro para o acesso a um barramento que funcione conforme descrito a seguir.
  - **Solução:**
    - Em um determinado momento, o circuito poderá então estar em um dentre os seguintes estados:
      - **Nenhum** dispositivo com acesso concedido ao barramento
      - Concessão de acesso para o **Dispositivo 1**
      - Concessão de acesso para o **Dispositivo 2**
    - Como são 3 os estados possíveis, precisamos de 2 variáveis de estado para codificar tais estados e, portanto, usaremos 2 flip-flops.
    - Com isso, podemos especificar as variáveis:
      - Entradas: $\mathbf{r_1}$ e $\mathbf{r_2}$
      - Saídas: $\mathbf{c_1}$ e $\mathbf{c_2}$
      - Variáveis de estados: $\mathbf{s_1}$ e $\mathbf{s_0}$ para estado atual e $\mathbf{n_1}$ e $\mathbf{n_0}$ para próximo estado.

8.1. Projeto de FSM (continuação)

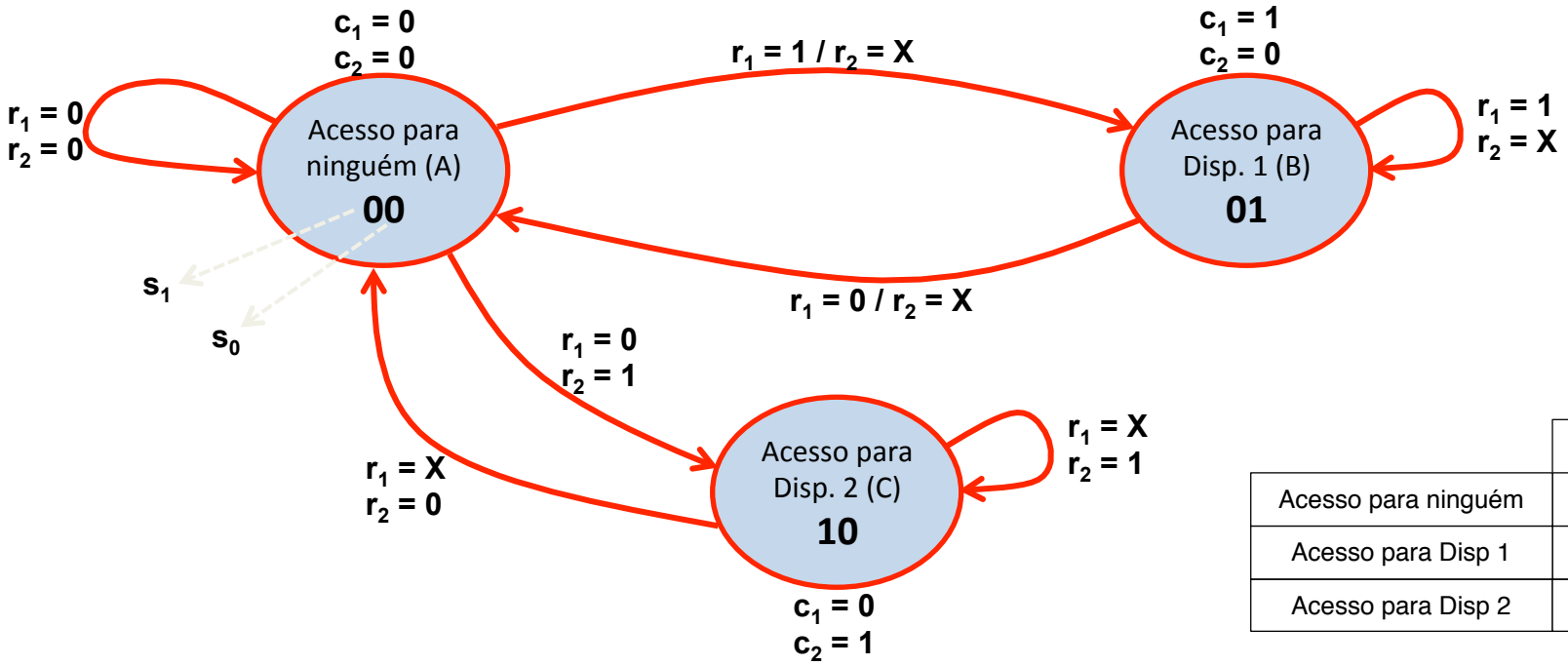
- **Exemplo 2**: Faça o projeto de um circuito árbitro para o acesso a um barramento que funcione conforme descrito a seguir.
  - **Solução:**
    - Construindo o diagrama de estados, temos:



	s1	s0
Acesso para ninguém (A)	0	0
Acesso para Disp 1 (B)	0	1
Acesso para Disp 2 (C)	1	0

8.1. Projeto de FSM (continuação)

- **Exemplo 2**: Faça o projeto de um circuito árbitro para o acesso a um barramento que funcione conforme descrito a seguir.
  - **Solução:**
    - Construindo o diagrama de estados, temos:



	s1	s0
Acesso para ninguém	0	0
Acesso para Disp 1	0	1
Acesso para Disp 2	1	0

8.1. Projeto de FSM (continuação)

- **Exemplo 2**: Faça o projeto de um circuito árbitro para o acesso a um barramento que funcione conforme descrito a seguir.
  - **Solução:**
    - Agora, a tabela:

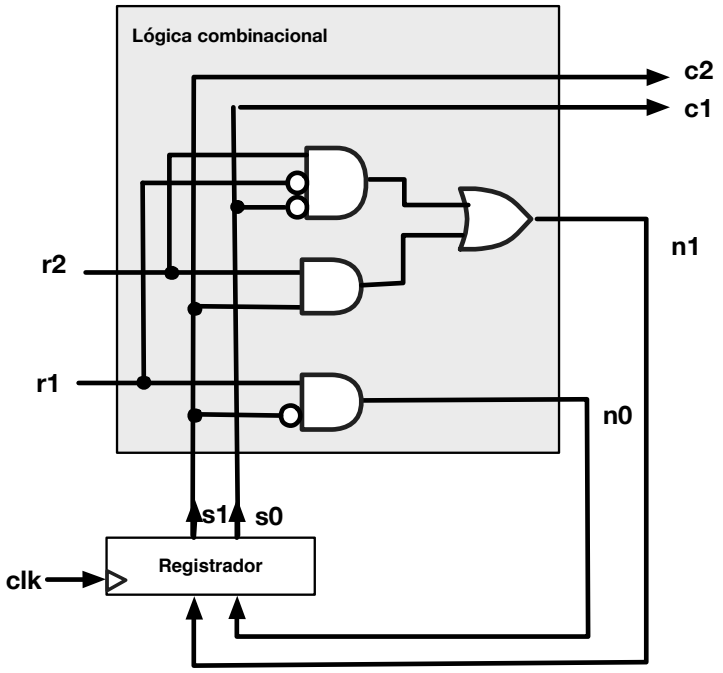
	Entradas				Saidas			
	s1	**s0**	**r1**	**r2**	**n1**	**n0**	**c1**	**c2**
A	0	0	0	0	0	0	0	0
	0	0	0	1	1	0	0	0
	0	0	1	—	0	1	0	0
B	0	1	0	—	0	0	1	0
	0	1	1	—	0	1	1	0
C	1	0	—	0	0	0	0	1
	1	0	—	1	1	0	0	1
	1	1	—	—	—	—	—	—

Don't cares podem ser usados para maior simplificação

Não usado (estado D)

8.1. Projeto de FSM (continuação)

- **Exemplo 2**: Faça o projeto de um circuito árbitro para o acesso a um barramento que funcione conforme descrito a seguir.
  - **Solução:**
    - E as equações:
      - $n_1 = s_1 r_2 + \overline{r}_1 r_2 \overline{s}_0$
      - $n_0 = r_1 \overline{s}_1$
      - $c_1 = s_0$
      - $c_2 = s_1$
    - Agora basta montar o circuito...



PROBLEMAS

Problema 8.1. Projete o circuito gerador da sequência (0001, 0011, 1100, 1000, repete...).

8. Projeto de Circuitos Sequenciais

8.1. Projeto de FSM (continuação)

8.2. FSM com estados de bloqueio (Lock-out)

8.3. Engenharia reversa de projeto FSM

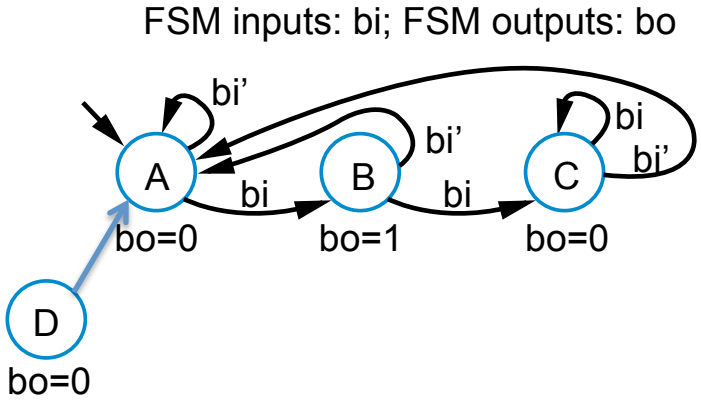
8.2. FSM com estados de bloqueio (Lock-out)

- **No exemplo 1:** Projetar um sincronizador de aperto de botão (circuito que converte um toque no botão em um pulso com duração de um ciclo de *clock*, independentemente do tempo que o botão fica apertado).

	s1	s0	bi	n1	n0	bo
A	0	0	0	0	0	0
	0	0	1	0	1	0
B	0	1	0	0	0	1
	0	1	1	1	0	1
C	1	0	0	0	0	0
	1	0	1	1	0	0
	1	1	0	—	—	—
	1	1	1	—	—	—

Não usado (estado D)

Don't cares podem ser preenchidos com "0"s



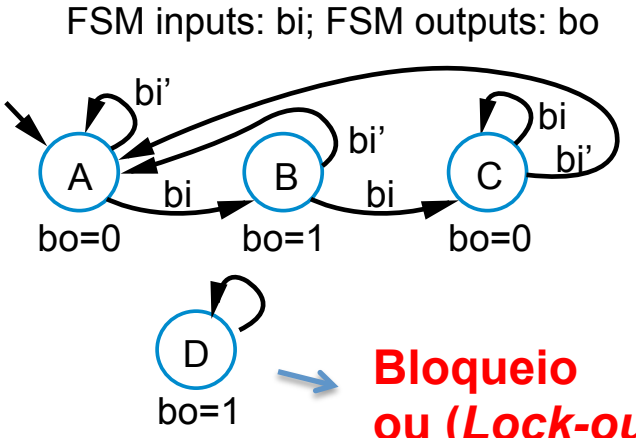
8.2. FSM com estados de bloqueio (Lock-out)

- **No exemplo 1:** Projetar um sincronizador de aperto de botão (circuito que converte um toque no botão em um pulso com duração de um ciclo de *clock*, independentemente do tempo que o botão fica apertado).

	s1	s0	bi	n1	n0	bo
A	0	0	0	0	0	0
	0	0	1	0	1	0
B	0	1	0	0	0	1
	0	1	1	1	0	1
C	1	0	0	0	0	0
	1	0	1	1	0	0
	1	1	0	—	—	—
	1	1	1	—	—	—

Não usado (estado D)

Don't cares podem ser preenchidos com "1"s



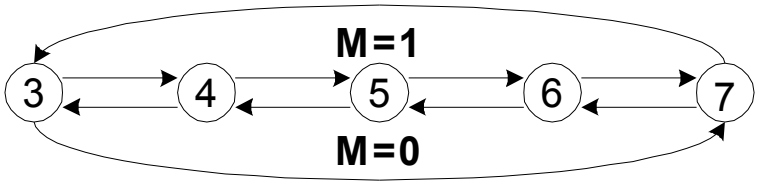
Bloqueio ou (*Lock-out*)

Como solucionar os estados de bloqueio?

8.2. FSM com estados de bloqueio (Lock-out)

- **Exemplo 3:** Projecto de um Contador Ascendente/Descendente Síncrono de Módulo 5 (PADM5)

Diagrama de Estados



Codificação de Estados

	s2	s0	s0
3	0	1	1
4	1	0	0
5	1	0	1
6	1	1	0
7	1	1	1

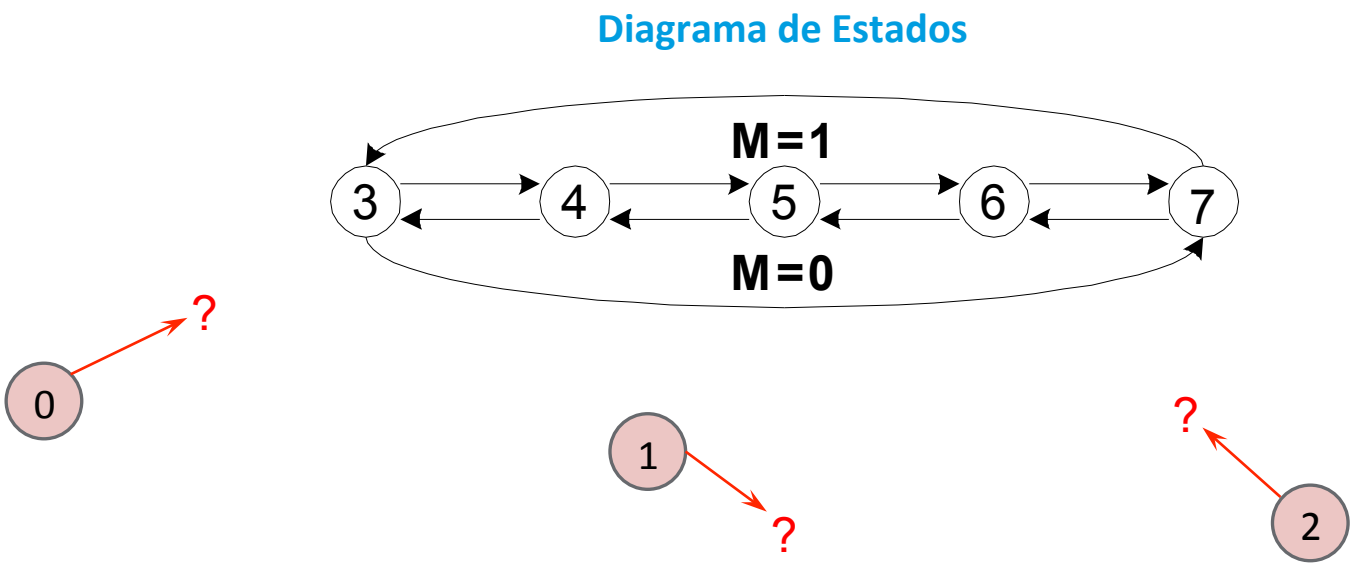
Tabela de Transição de Estados

	Entradas				Saidas		
	s2	**s1**	**s0**	**M**	**n2**	**n1**	**n0**
3	0	1	1	0	1	1	1
	0	1	1	1	1	0	0
4	1	0	0	0	0	1	1
	1	0	0	1	1	0	1
5	1	0	1	0	1	0	0
	1	0	1	1	1	1	0
6	1	1	0	0	1	0	1
	1	1	0	1	1	1	1
7	1	1	1	0	1	1	0
	1	1	1	1	0	1	1
	0	0	0	–	–	–	–
	0	0	1	–	–	–	–
	0	1	0	–	–	–	–

Não usados (estado 0, 1 e 2)

8.2. FSM com estados de bloqueio (Lock-out)

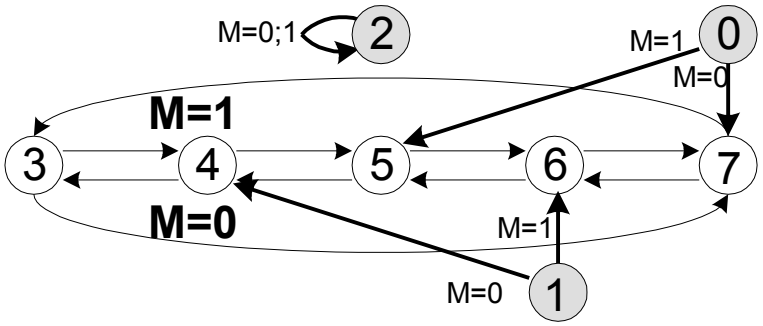
- **Exemplo 3:** O que acontece se a máquina transitar para um estado fora da gama prevista (lock-out)?



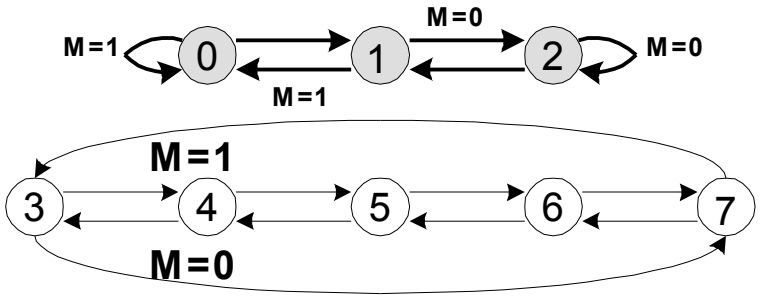
8.2. FSM com estados de bloqueio (Lock-out)

- "Estados de LOCK-OUT: no caso de não serem utilizados todos os estados disponíveis, pode ocorrer a situação do contador se encontrar num estado não desejado (fora da sequência de contagem) devido a ruído no circuito ou à não imposição de estado inicial.
- Nessa situação ou o contador entra na sequência de contagem pretendida ou fica indefinidamente no exterior (Lock-Out).

Exemplo de Lock-Out (1)



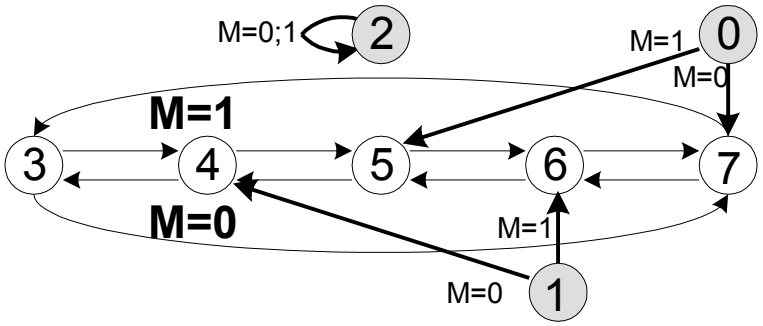
Exemplo de Lock-Out (2)



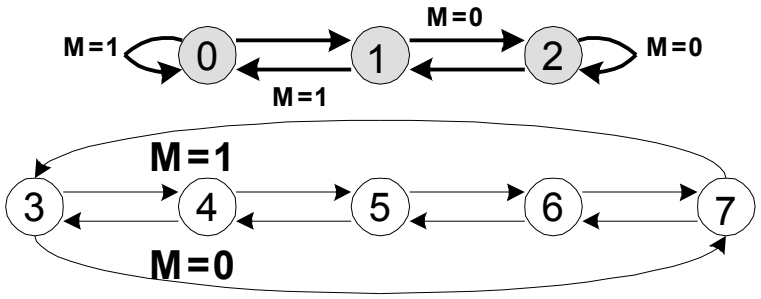
8.2. FSM com estados de bloqueio (Lock-out)

- **Solução 1:** impor a transição de qualquer estado externo para um estado da sequência de contagem
- **Solução 2:** considerar uma entrada extra, de inicialização, que coloque o sistema num dos estados de contagem pretendido.

Exemplo de Lock-Out (1)

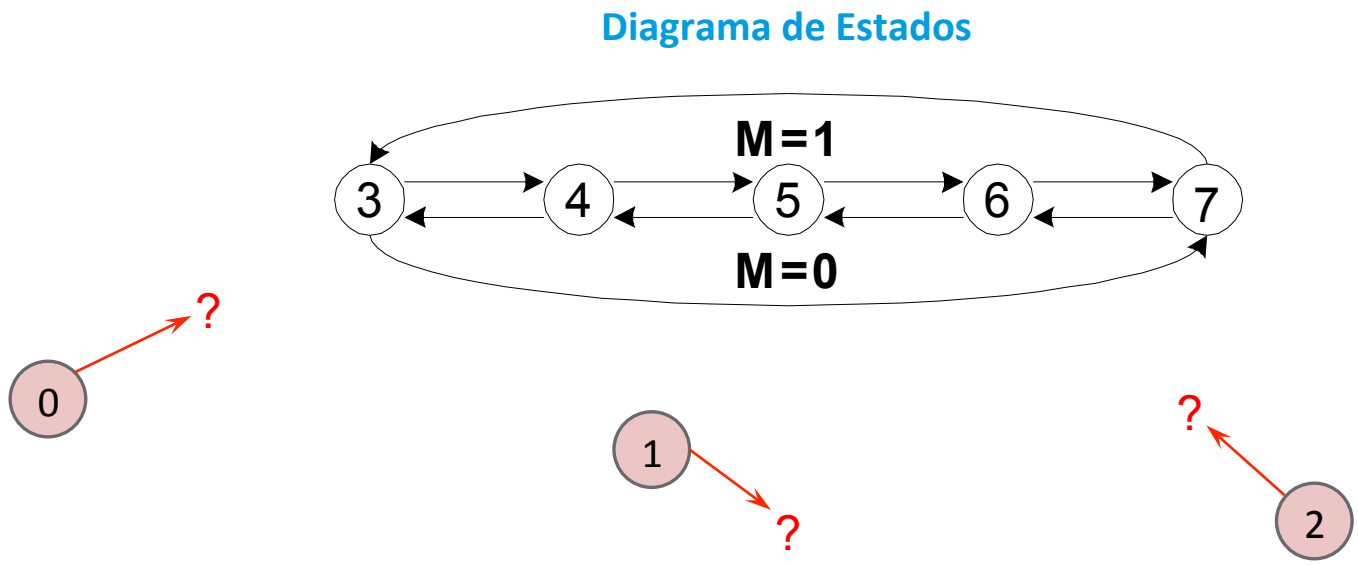


Exemplo de Lock-Out (2)



8.2. FSM com estados de bloqueio (Lock-out)

- **Exemplo 3:** O que acontece se a máquina transitar para um estado fora da gama prevista (lock-out)?



8.2. FSM com estados de bloqueio (Lock-out)

- **Exemplo 3:** O que acontece se a máquina transitar para um estado fora da gama prevista (lock-out)?

$$n_2 = \left(\overline{M} + \overline{s}_2 + \overline{s}_1 + \overline{s}_0 \right) \cdot \left(M + s_1 + s_0 \right)$$

$$n_1 = \overline{M}\,\overline{s}_1\,\overline{s}_0 + M\,\overline{s}_1\,s_0 + \overline{M}\,s_1\,s_0 + M\,s_2\,s_1$$

$$n_0 = M\,s_2\,s_1 + \overline{s}_0 + \overline{M}\,\overline{s}_2$$

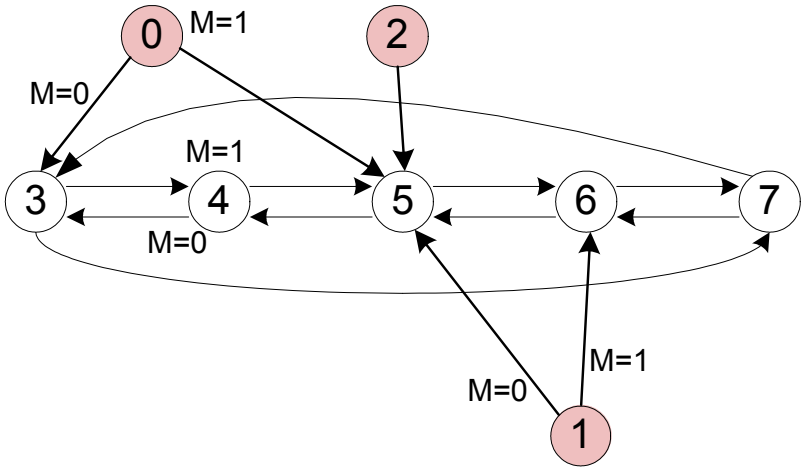
Estados fora da gama prevista definidos para evitar *lock-outs*

Tabela de Transição de Estados

	Entradas				Saidas		
	s2	**s1**	**s0**	**M**	**n2**	**n1**	**n0**
3	0	1	1	0	1	1	1
	0	1	1	1	1	0	0
4	1	0	0	0	0	1	1
	1	0	0	1	1	0	1
5	1	0	1	0	1	0	0
	1	0	1	1	1	1	0
6	1	1	0	0	1	0	1
	1	1	0	1	1	1	1
7	1	1	1	0	1	1	0
	1	1	1	1	0	1	1
0	0	0	0	0	0	1	1
	0	0	0	1	1	0	1
1	0	0	1	0	1	0	1
	0	0	1	1	1	1	0
2	0	1	0	–	1	0	1

8.2. FSM com estados de bloqueio (Lock-out)

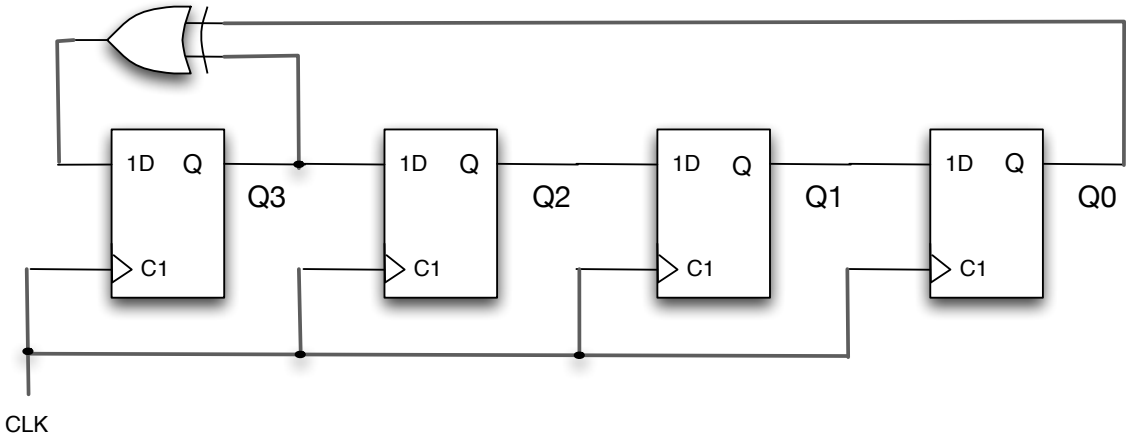
- **Exemplo 3:** O que acontece se a máquina transitar para um estado fora da gama prevista (lock-out)?



Os estados, não considerados para efeito da contagem (estados 0, 1 e 2), permitem passar a máquina para a sequência de contagem pretendida ao fim de um ciclo de relógio.

PROBLEMAS

Problema 8.2. Considere o circuito da figura e obtenha a sequencia que gera partindo do estado inicial Q3Q2Q1Q0=1000. Utilizando portas lógicas modifique o circuito de forma de evitar situações de bloqueio (*lock-outs)*.



QUESTÃO 1: Tempo 5 min

8. Projeto de Circuitos Sequenciais

8.1. Projeto de FSM (continuação)

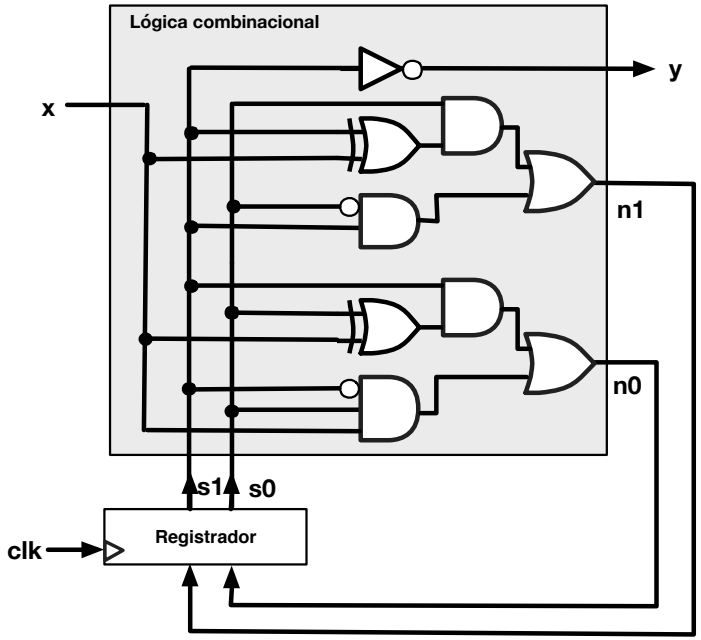
8.2. FSM com estados de bloqueio (Lock-out)

8.3. Engenharia reversa de projeto FSM

8.3. Engenharia reversa de projeto FSM

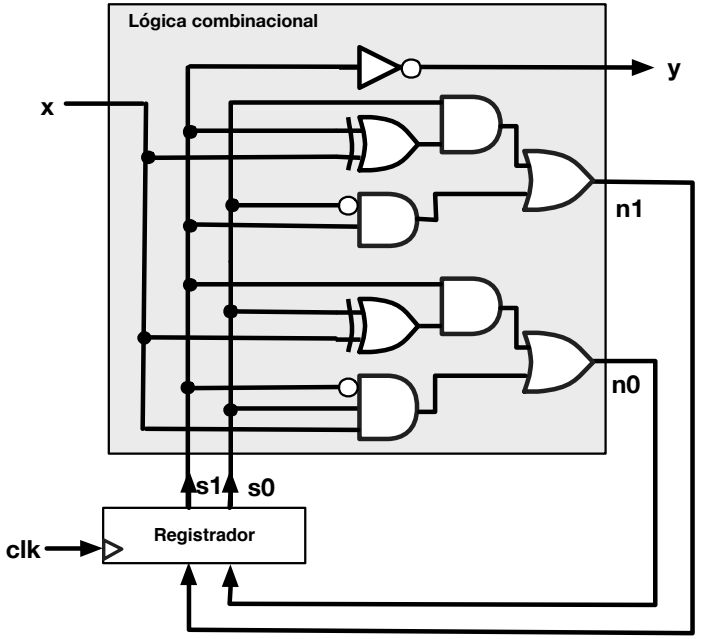
- Engenharia reversa de projeto FSM consiste em, a partir do circuito obter o diagrama de estados.

- **Exemplo 4:** Projete o diagrama de estados associado ao circuito sequencial.



8.3. Engenharia reversa de projeto FSM

- **Exemplo 4:** Projete o diagrama de estados associado ao circuito sequencial.



$$n_0 = \overline{s_1} s_0 x + (x \oplus s_1) s_0$$

$$n_1 = s_1 \overline{s_0} + (x \oplus s_0) s_1$$

$$y = \overline{s_1}$$

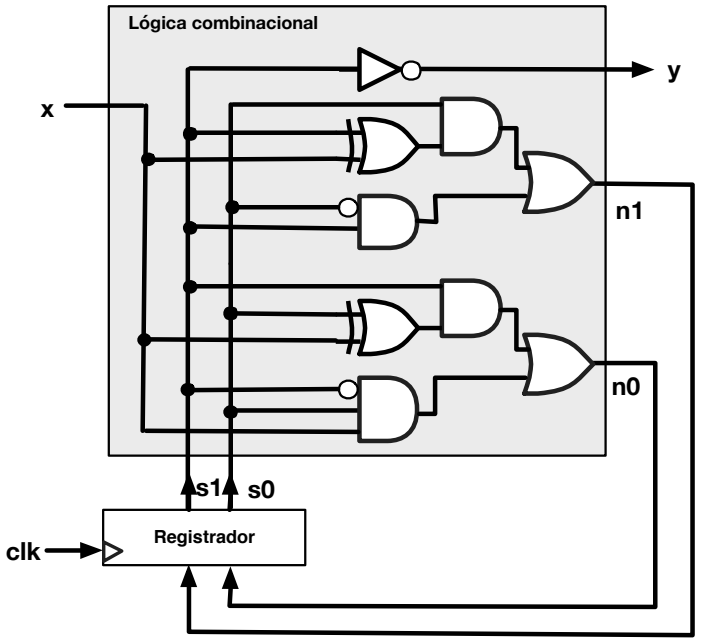
1. Obter funções lógicas

8.3. Engenharia reversa de projeto FSM

- **Exemplo 4:** Projete o diagrama de estados associado ao circuito sequencial.

2. Obter Tabela de transição de estados

s1	s0	x	n1	n0	y
0	0	0	0	0	1
0	0	1	0	0	1
0	1	0	0	0	1
0	1	1	1	1	1
1	0	0	1	0	0
1	0	1	1	1	0
1	1	0	1	1	0
1	1	1	0	0	0



$$n_0 = \overline{s_1} s_0 x + (x \oplus s_1) s_0$$

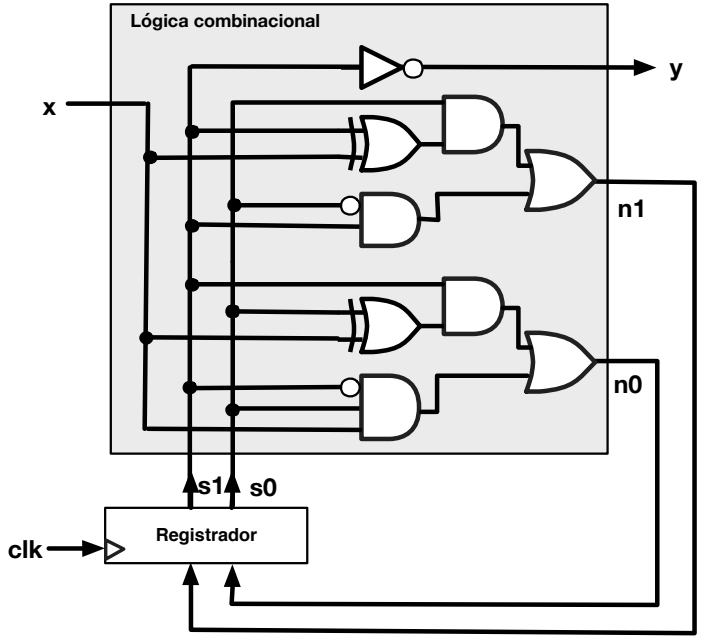
$$n_1 = s_1 \overline{s_0} + (x \oplus s_0) s_1$$

$$y = \overline{s_1}$$

Obter funções lógicas

8.3. Engenharia reversa de projeto FSM

- **Exemplo 4:** Projete o diagrama de estados associado ao circuito sequencial.



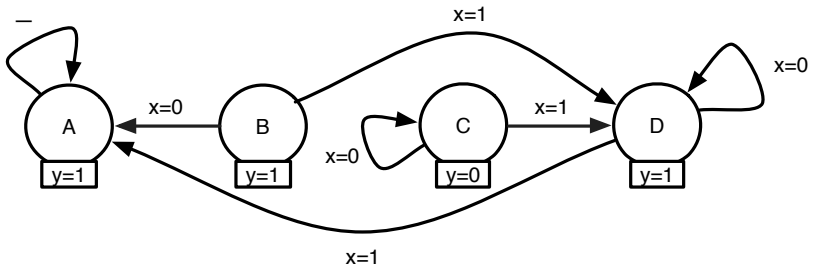
2. Obter Tabela de transição de estados

ESTADOS

A	C
B	D

s1	s0	x	n1	n0	y
0	0	0	0	0	1
0	0	1	0	0	1
0	1	0	0	0	1
0	1	1	1	1	1
1	0	0	1	0	0
1	0	1	1	1	0
1	1	0	1	1	0
1	1	1	0	0	0

3. Decodificar os estados



4. Obter diagrama de estados

$$n_0 = \overline{s_1} s_0 x + (x \oplus s_1) s_0$$

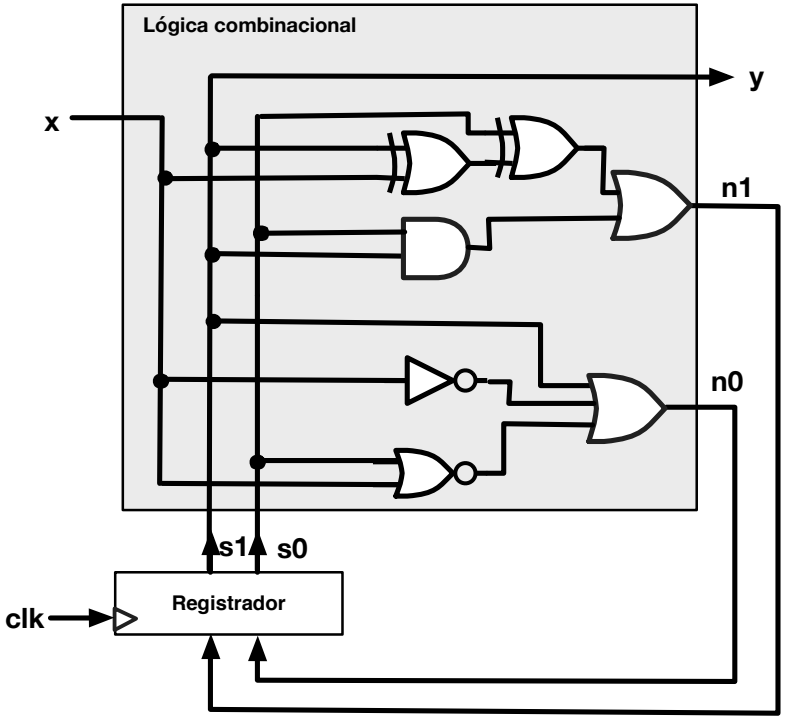
$$n_1 = s_1 \overline{s_0} + (x \oplus s_0) s_1$$

$$y = \overline{s_1}$$

1. Obter funções lógicas

PROBLEMAS

Problema 8.3. Projete o diagrama de estados associado ao circuito sequencial.



QUESTÃO 2: Tempo 5 min



FEDERAL UNIVERSITY
OF SANTA CATARINA

EEL5105 – Circuitos e Técnicas Digitais
Aula 8

Prof. Héctor Pettenghi

hector@eel.ufsc.br
http://hectorpettenghi.paginas.ufsc.br

Material desenvolvido com apoio de arquivos de apresentação do livro de Frank Vahid

Exercícios

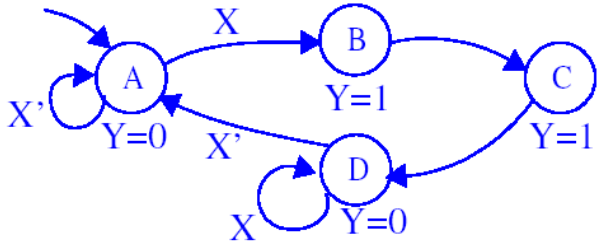
- Exercícios do Livro do **Frank Vahid**
  - **3.23** até **3.27**
  - **3.29** e **3.30**
  - **3.32** e **3.33**
  - **3.38** até **3.42**
- **A versão digital do livro do Frank Vahid está disponível no site da BU**
  - Mais especificamente em: http://150.162.4.10/pergamum/biblioteca_s/php/login_pearson.php

Exercícios

- Respostas Livro do **Frank Vahid**

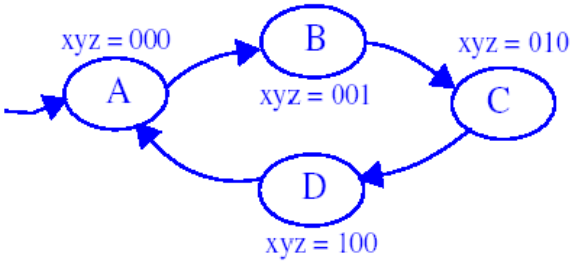
3.23

Inputs: X, *Outputs*: Y



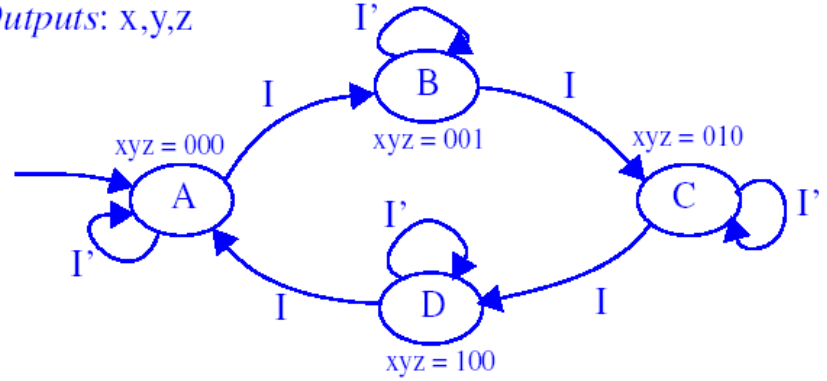
3.24

Inputs: None, *Outputs*: x,y,z



3.25

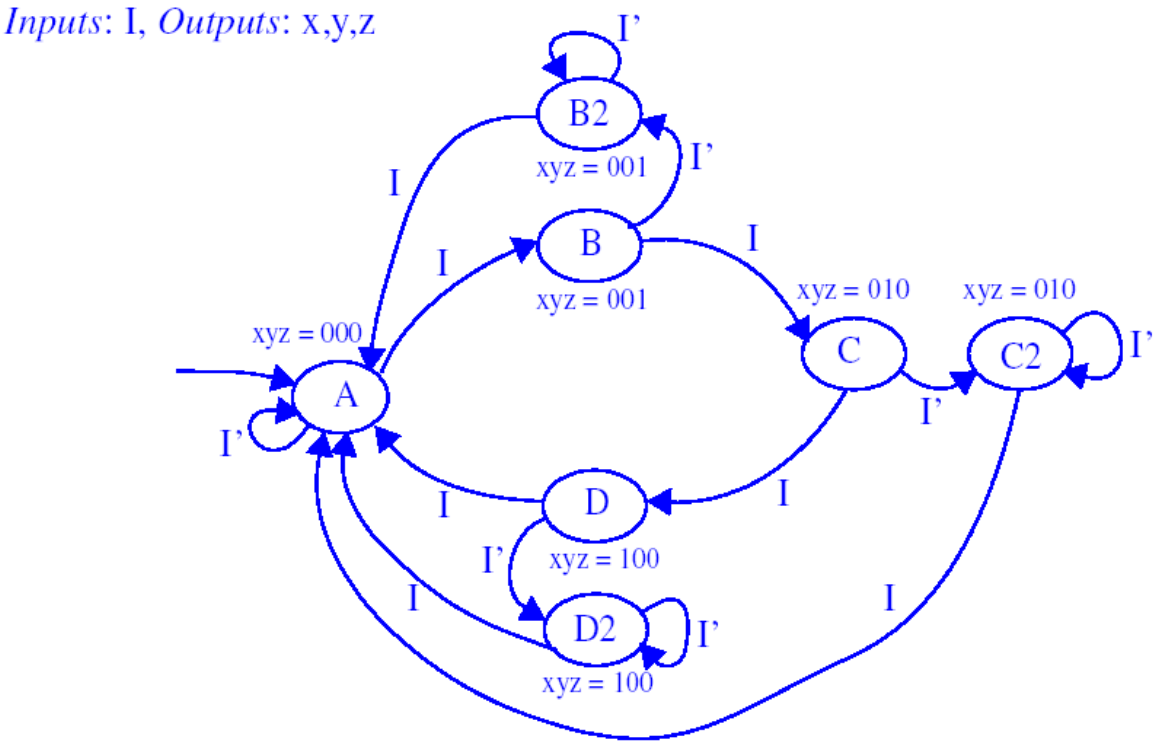
Inputs: I, *Outputs*: x,y,z



Exercícios

- Respostas Livro do **Frank Vahid**

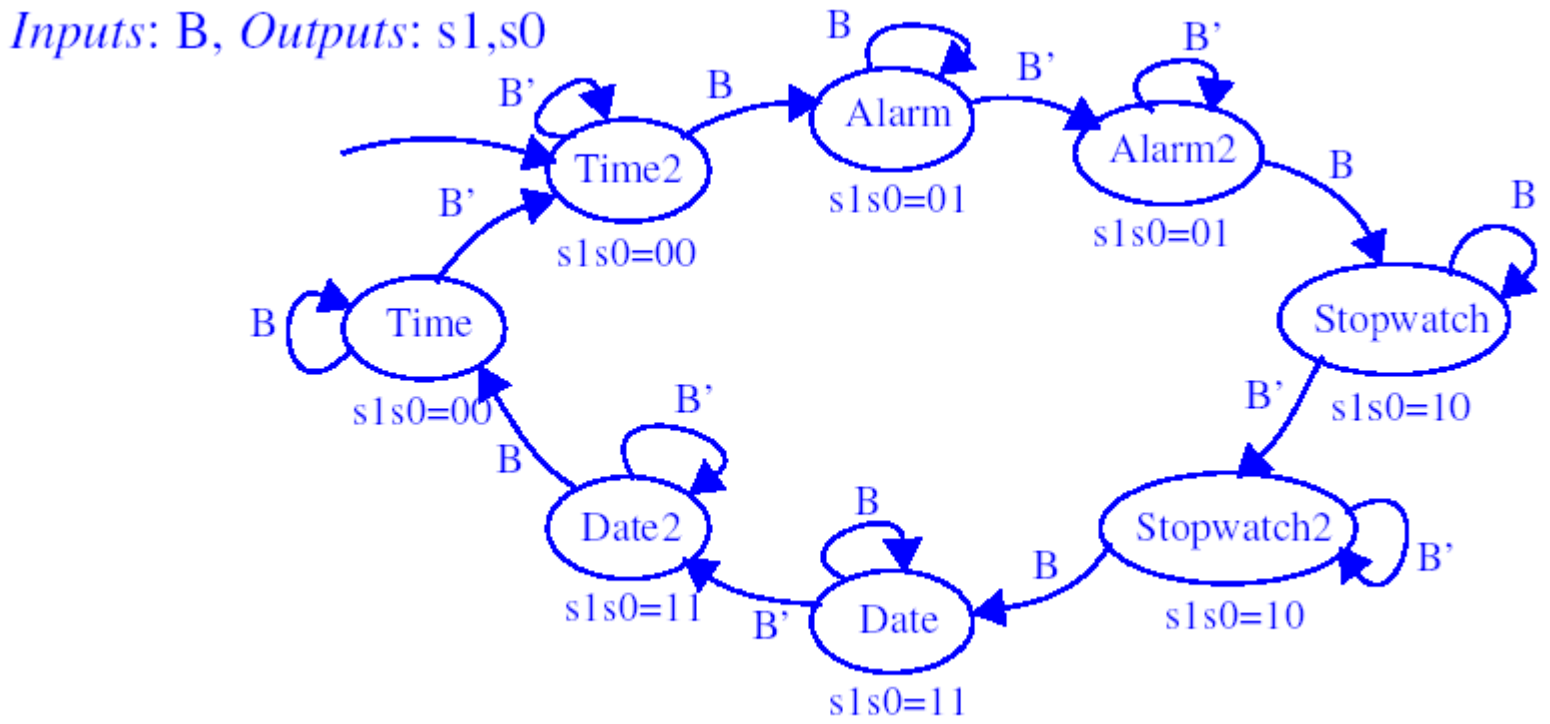
3.26



Exercícios

- Respostas Livro do **Frank Vahid**

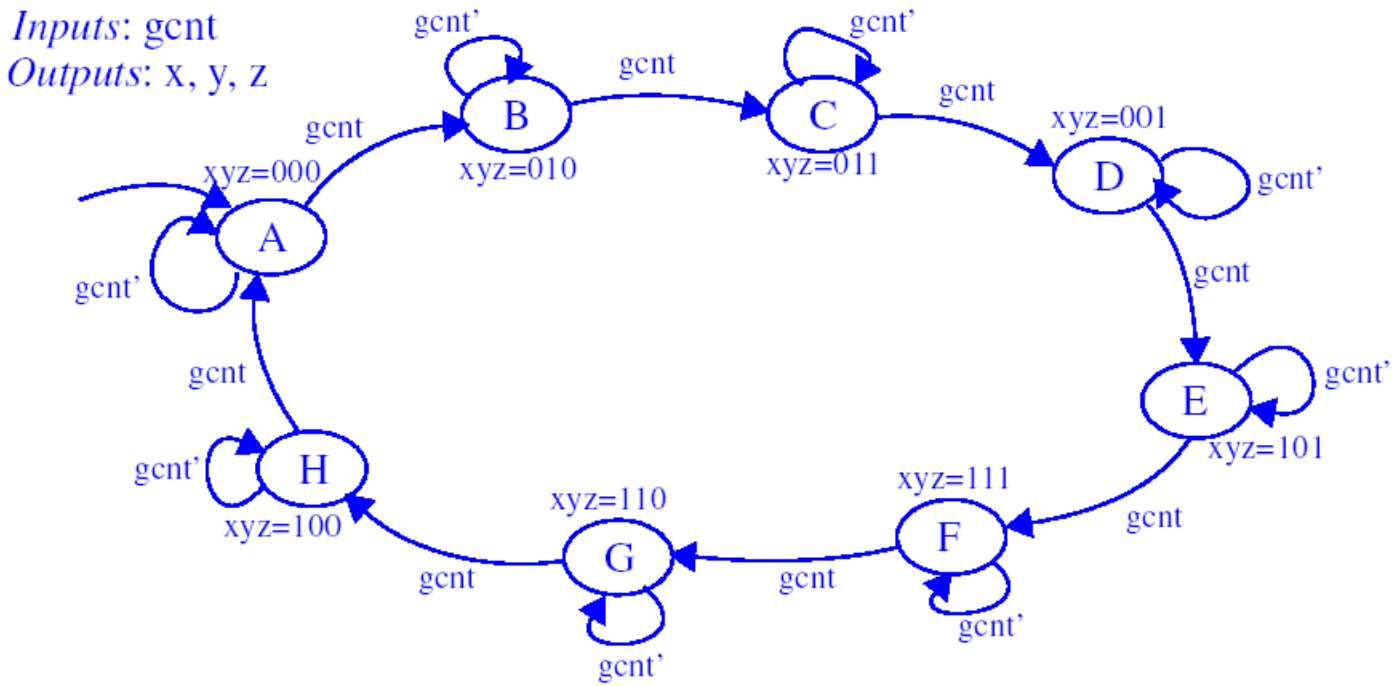
3.27



Exercícios

- Respostas Livro do **Frank Vahid**

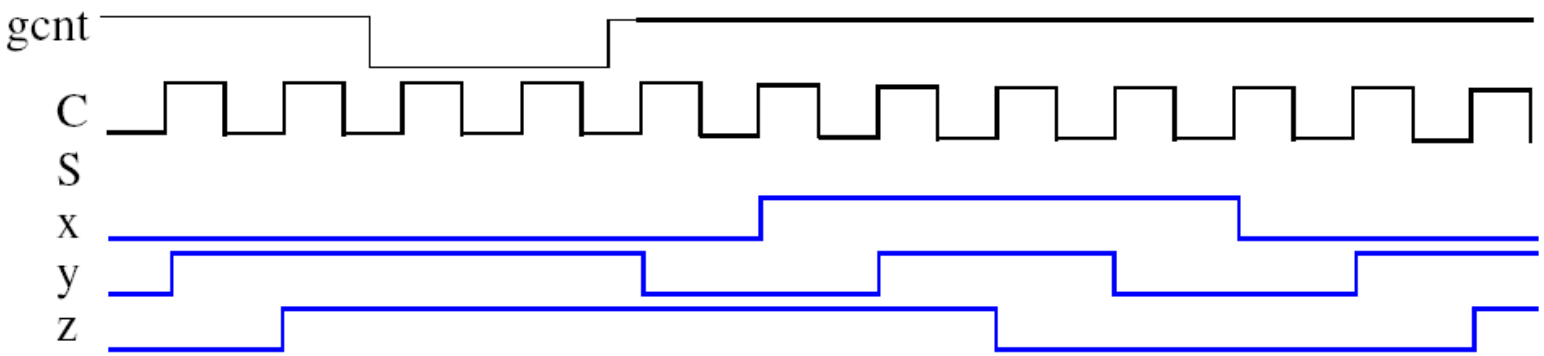
3.29



Exercícios

- Respostas Livro do **Frank Vahid**

3.30



Exercícios

- Respostas Livro do **Frank Vahid**

3.32

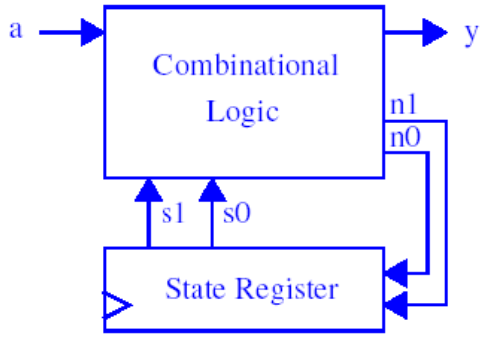
a) 2 bits
b) 3 bits
c) 4 bits
d) 5 bits
e) 10 bits

3.33

$2^{16} = 65{,}536$ possible states

Exercícios

- Resposta do **3.38**



A straightforward encoding is A=00, B=01, C=10, D=11.

Inputs			Outputs		
s1	s0	a	n1	n0	y
0	0	0	0	1	0
0	0	1	0	0	0
0	1	0	0	1	1
0	1	1	1	0	1
1	0	0	1	1	1
1	0	1	1	1	1
1	1	0	0	0	0
1	1	1	0	0	0

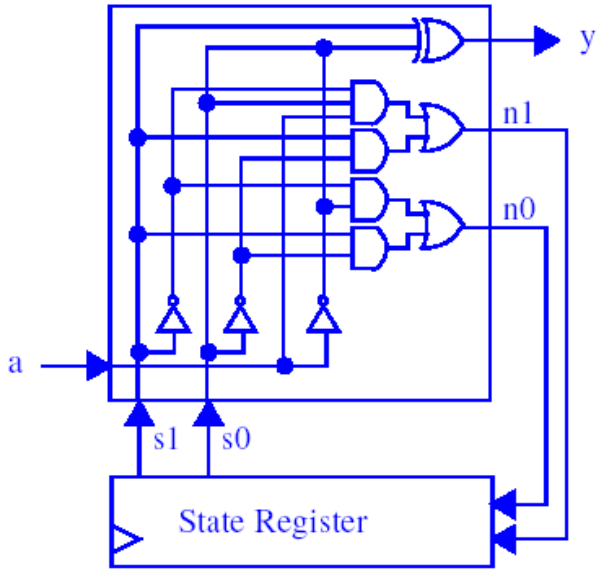
Exercícios

- Resposta do **3.38**

$n1 = s1's0a + s1s0'a' + s1s0'a = s1's0a + s1s0'$

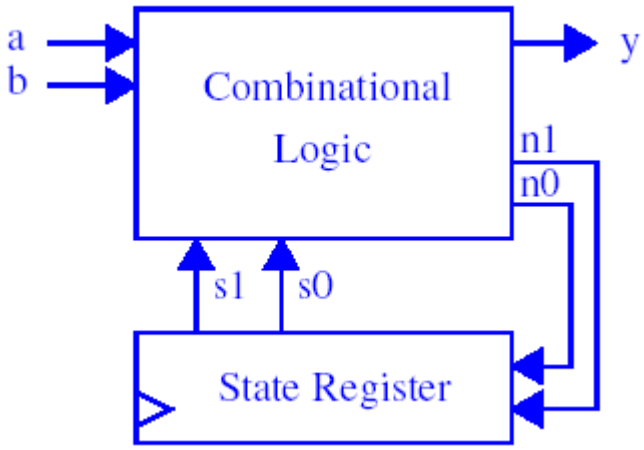
$n0 = s1's0'a' + s1's0a' + s1s0'a' + s1s0'a = s1'a' + s1s0'$

$y = s1's0a' + s1's0a + s1s0'a' + s1s0'a = s1's0 + s1s0' = s1 \text{ xor } s0$



Exercícios

- Resposta do **3.39**



Inputs				Outputs		
s1	s0	a	b	n1	n0	y
0	0	0	0	1	0	0
0	0	0	1	0	1	0
0	0	1	0	0	0	0
0	0	1	1	0	0	0
0	1	0	0	0	1	1
0	1	0	1	0	1	1
0	1	1	0	1	0	1
0	1	1	1	1	0	1
1	0	0	0	1	0	1
1	0	0	1	1	1	1
1	0	1	0	1	0	1
1	0	1	1	1	1	1
1	1	0	0	0	0	0
1	1	0	1	0	0	0
1	1	1	0	0	0	0
1	1	1	1	0	0	0

A straightforward encoding is A=00, B=01, C=10, D=11.

Exercícios

- Resposta do **3.39**

Step 5 - Implement the combinational logic

n1 = s1's0'a'b' + s1's0a + s1s0'

n0 = s1's0'a'b + s1's0a' + s1s0'b

y = s1's0 + s1s0'

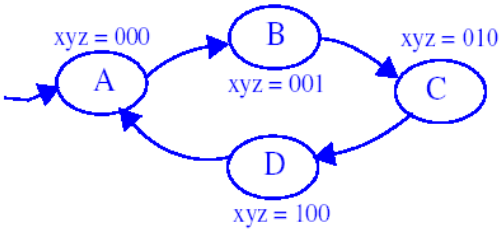
Note: The above equations can be minimized further.

Exercícios

- Resposta do **3.40**

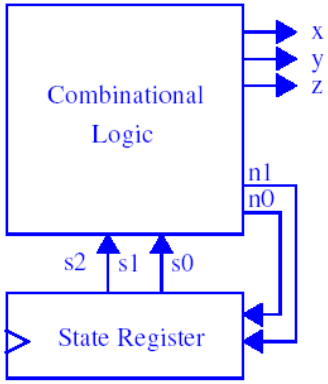
Step 1 - Capture the FSM

Inputs: None, *Outputs*: x,y,z



The FSM was created during Exercise 3.24.

Step 2 - Create the architecture



Step 3 - Encode the states

A straightforward encoding is A=00, B=01, C=10, D=11.

Step 4 - Create the state table

Inputs		Outputs				
s1	s0	n1	n0	x	y	z
0	0	0	1	0	0	0
0	1	1	0	0	0	1
1	0	1	1	0	1	0
1	1	0	0	1	0	0

Step 5 - Implement the combinational logic

n1 = s1's0 + s1s0' = s1 XOR s0

n0 = s1's0' + s1s0' = s0'

x = s1s0

y = s1s0'

z = s1's0